Recurrent Neural Networks

A Mathematical Treatment

Foundations of Sequential Learning:

Architecture, Backpropagation Through Time,

and the Vanishing Gradient Problem

January 25, 2026

Abstract

This document provides a rigorous mathematical treatment of Recurrent Neural Networks (RNNs). We develop the theoretical foundations of recurrent architectures, including the complete derivation of the forward propagation equations and the Backpropagation Through Time (BPTT) algorithm. A central focus is the formal analysis of the vanishing and exploding gradient problem, which motivates the development of gated architectures such as LSTMs. This treatment serves as the mathematical prerequisite for understanding modern recurrent architectures.

Contents

1 Introduction **3**
1.1 Motivation: Sequential Data . . . 3
1.2 The Recurrence Principle . . . 3

2 Mathematical Preliminaries **3**
2.1 Notation . . . 3
2.2 Activation Functions . . . 4
2.3 Vector Calculus Conventions . . . 4

3 RNN Architecture **5**
3.1 The Elman Network . . . 5
3.2 Network Architecture Diagrams . . . 5
3.3 Unfolding in Time . . . 5
3.4 Parameter Sharing . . . 6
3.5 Parameter Count . . . 6

4 Loss Functions **6**
4.1 Sequence Loss . . . 6
4.2 Common Loss Functions . . . 7

5 Backpropagation Through Time **7**
5.1 Overview . . . 8
5.2 Error Signal Definitions . . . 8
5.3 BPTT Derivation . . . 8
5.4 Weight Gradients . . . 9
5.5 Complete BPTT Algorithm . . . 10

6 The Vanishing and Exploding Gradient Problem **10**
6.1 Gradient Flow Analysis . . . 10
6.2 The Vanishing Gradient Problem . . . 11
6.3 The Exploding Gradient Problem . . . 11
6.4 Formal Characterization . . . 12

7 Mitigation Strategies **13**
7.1 Gradient Clipping . . . 13
7.2 Orthogonal Initialization . . . 13
7.3 Truncated BPTT . . . 13
7.4 Architectural Solutions: Gated RNNs . . . 13

8 RNN Variants **13**
8.1 Jordan Network 13
8.2 Deep RNN 14
8.3 Bidirectional RNN 14

9 Conclusion **14**

1 Introduction

1.1 Motivation: Sequential Data

Many real-world problems involve sequential data where the ordering of elements carries essential information. Examples include natural language (sequences of words), speech (sequences of audio frames), time series (sequences of measurements), and video (sequences of frames).

Definition 1.1 (Sequence)**.** A sequence of length T over a set $\mathcal{X}$ is an ordered tuple:

$$\mathbf{x}_{1:T} = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T), \quad \mathbf{x}_t \in \mathcal{X} \text{ for all } t \in \{1, \ldots, T\} \tag{1}$$

The fundamental challenge in sequential modeling is to learn a function that captures dependencies between elements at different positions in the sequence. Standard feedforward networks treat inputs as independent, making them unsuitable for this task.

1.2 The Recurrence Principle

The key insight of recurrent neural networks is to maintain a *hidden state* that summarizes information from all previous time steps.

Definition 1.2 (Recurrence Relation)**.** A recurrence relation defines the hidden state $\mathbf{h}_t$ as a function of the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \boldsymbol{\theta}) \tag{2}$$

where f is a parameterized function and $\boldsymbol{\theta}$ represents the learnable parameters.

This recursive structure allows the network to, in principle, capture dependencies of arbitrary length while using a fixed number of parameters.

2 Mathematical Preliminaries

2.1 Notation

We establish the following notation used throughout this document:

Symbol	Description
n	Dimension of input vectors
m	Dimension of hidden state vectors
k	Dimension of output vectors
T	Sequence length
$\mathbf{x}_t \in \mathbb{R}^n$	Input vector at time step t
$\mathbf{h}_t \in \mathbb{R}^m$	Hidden state vector at time step t
$\mathbf{y}_t \in \mathbb{R}^k$	Output vector at time step t
$\hat{\mathbf{y}}_t \in \mathbb{R}^k$	Target vector at time step t
$\mathbf{W} \in \mathbb{R}^{m \times n}$	Input-to-hidden weight matrix
$\mathbf{U} \in \mathbb{R}^{m \times m}$	Hidden-to-hidden (recurrent) weight matrix
$\mathbf{V} \in \mathbb{R}^{k \times m}$	Hidden-to-output weight matrix
$\mathbf{b} \in \mathbb{R}^m$	Hidden layer bias vector
$\mathbf{c} \in \mathbb{R}^k$	Output layer bias vector

Table 1: Notation conventions

2.2 Activation Functions

Definition 2.1 (Hyperbolic Tangent)**.** The hyperbolic tangent function $\tanh : \mathbb{R} \to (-1, 1)$ is defined as:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \tag{3}$$

Applied element-wise to vectors: $[\tanh(\mathbf{z})]_i = \tanh(z_i)$.

Proposition 2.2 (Derivative of Tanh)**.** *The derivative of* tanh *is:*

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z) = \operatorname{sech}^2(z) \tag{4}$$

Note that $\frac{d}{dz}\tanh(z) \in (0, 1]$ *with equality at* $z = 0$.

Proof. Let $y = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$. Using the quotient rule:

$$\begin{aligned}
\frac{dy}{dz} &= \frac{(e^z + e^{-z})(e^z + e^{-z}) - (e^z - e^{-z})(e^z - e^{-z})}{(e^z + e^{-z})^2} \\
&= \frac{(e^z + e^{-z})^2 - (e^z - e^{-z})^2}{(e^z + e^{-z})^2} \\
&= 1 - \left(\frac{e^z - e^{-z}}{e^z + e^{-z}}\right)^2 = 1 - \tanh^2(z)
\end{aligned}$$

□

Definition 2.3 (Softmax Function)**.** For classification tasks, the softmax function $\text{softmax} : \mathbb{R}^k \to \Delta^{k-1}$ maps vectors to probability distributions:

$$[\text{softmax}(\mathbf{z})]_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}, \quad i = 1, \ldots, k \tag{5}$$

where Δ^{k-1} denotes the $(k-1)$-dimensional probability simplex.

Proposition 2.4 (Softmax Jacobian)**.** *The Jacobian matrix of the softmax function is:*

$$\frac{\partial[\text{softmax}(\mathbf{z})]_i}{\partial z_j} = [\text{softmax}(\mathbf{z})]_i \left(\delta_{ij} - [\text{softmax}(\mathbf{z})]_j\right) \tag{6}$$

where δ_{ij} *is the Kronecker delta.*

2.3 Vector Calculus Conventions

Definition 2.5 (Jacobian Matrix)**.** For a function $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian matrix $\mathbf{J} \in \mathbb{R}^{m \times n}$ is:

$$J_{ij} = \frac{\partial f_i}{\partial x_j} \tag{7}$$

Definition 2.6 (Gradient of Scalar Function)**.** For a scalar function $L : \mathbb{R}^n \to \mathbb{R}$, the gradient is the column vector:

$$\nabla_{\mathbf{x}} L = \left[\frac{\partial L}{\partial x_1}, \ldots, \frac{\partial L}{\partial x_n}\right]^\top \in \mathbb{R}^n \tag{8}$$

Lemma 2.7 (Chain Rule for Vectors)**.** *For composed functions* $L = g(\mathbf{f}(\mathbf{x}))$ *where* $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$ *and* $g : \mathbb{R}^m \to \mathbb{R}$*:*

$$\nabla_{\mathbf{x}} L = \mathbf{J}_{\mathbf{f}}^\top \nabla_{\mathbf{f}} g \tag{9}$$

where $\mathbf{J}_{\mathbf{f}}$ *is the Jacobian of* $\mathbf{f}$.

3 RNN Architecture

3.1 The Elman Network

The standard RNN architecture, also known as the Elman network, is defined by the following equations.

Definition 3.1 (Vanilla RNN)**.** A vanilla recurrent neural network is defined by the system:

$$\mathbf{a}_t = \mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b} \tag{10}$$
$$\mathbf{h}_t = \tanh(\mathbf{a}_t) \tag{11}$$
$$\mathbf{o}_t = \mathbf{V}\mathbf{h}_t + \mathbf{c} \tag{12}$$
$$\mathbf{y}_t = g(\mathbf{o}_t) \tag{13}$$

where g is an output activation function (identity for regression, softmax for classification).

RNN Forward Equations

$$\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}) \qquad \text{[Hidden State Update]} \tag{14}$$
$$\mathbf{y}_t = g(\mathbf{V}\mathbf{h}_t + \mathbf{c}) \qquad \text{[Output Computation]} \tag{15}$$

with initial condition $\mathbf{h}_0 = \mathbf{0} \in \mathbb{R}^m$ (or learned).

3.2 Network Architecture Diagrams

The RNN architecture can be visualized in two equivalent forms: the *folded* (compact) representation showing the recurrent connection, and the *unfolded* representation showing the computational graph across time.

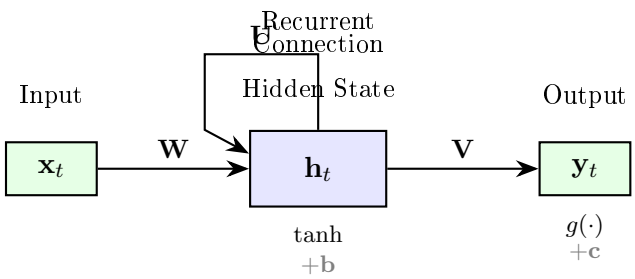


Figure 1: Folded (compact) representation of the RNN. The recurrent connection from $\mathbf{h}_t$ back to itself (via $\mathbf{U}$) represents the temporal dependency.

3.3 Unfolding in Time

The recurrence relation can be "unfolded" into an equivalent feedforward computation graph.

Definition 3.2 (Unfolded RNN)**.** For a sequence of length T, the unfolded RNN is the composition:

$$\mathbf{h}_T = f(f(\cdots f(f(\mathbf{h}_0, \mathbf{x}_1), \mathbf{x}_2) \cdots, \mathbf{x}_{T-1}), \mathbf{x}_T) \tag{16}$$

This can be visualized as a deep feedforward network with T layers, where all layers share the same parameters.

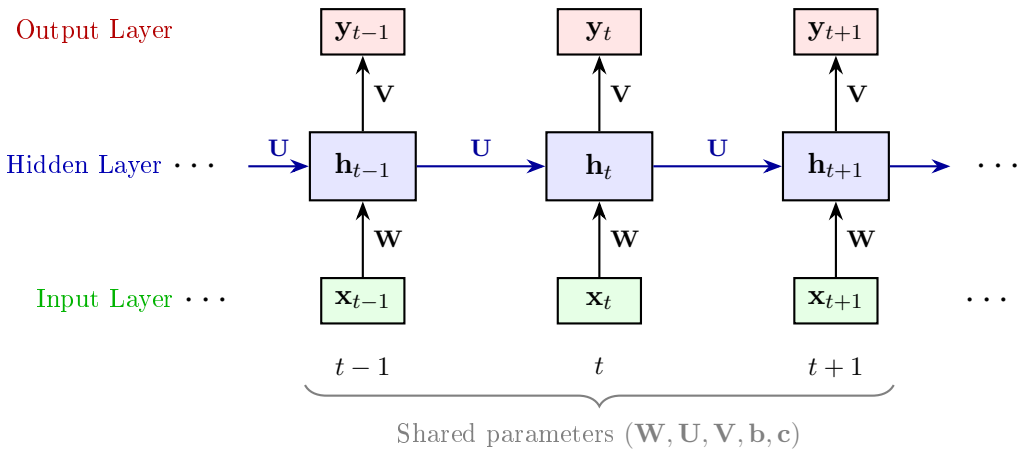


Figure 2: Unfolded representation of the RNN across three time steps. The same parameters are shared at every time step. Blue horizontal arrows show the flow of hidden state information through time.

Proposition 3.3 (Functional Form of Hidden State)**.** *The hidden state at time t can be expressed as a function of all previous inputs:*

$$\mathbf{h}_t = F_t(\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_t; \boldsymbol{\theta}) \tag{17}$$

where F_t is implicitly defined by the recurrence. This demonstrates that $\mathbf{h}_t$ contains (compressed) information about the entire history.

3.4 Parameter Sharing

A crucial property of RNNs is *parameter sharing* across time steps.

Remark 3.4 (Parameter Sharing)**.** The same parameters $(\mathbf{W}, \mathbf{U}, \mathbf{V}, \mathbf{b}, \mathbf{c})$ are used at every time step. This provides:

(i) **Generalization**: The network can process sequences of any length.

(ii) **Efficiency**: The number of parameters is independent of sequence length.

(iii) **Statistical sharing**: Patterns learned at one position transfer to others.

3.5 Parameter Count

Proposition 3.5 (RNN Parameter Count)**.** *For an RNN with input dimension n, hidden dimension m, and output dimension k, the total number of parameters is:*

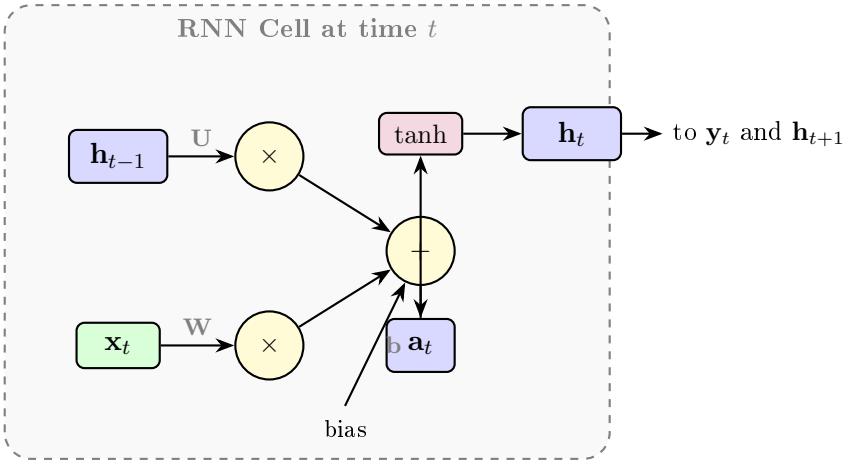
$$P = mn + m^2 + m + km + k = m(n + m + 1) + k(m + 1) \tag{18}$$

4 Loss Functions

4.1 Sequence Loss

Definition 4.1 (Total Sequence Loss)**.** For a sequence with targets $\hat{\mathbf{y}}_{1:T}$, the total loss is the sum of per-timestep losses:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{t=1}^{T} \mathcal{L}_t(\mathbf{y}_t, \hat{\mathbf{y}}_t) \tag{19}$$



$$\mathbf{a}_t = \mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}$$
$$\mathbf{h}_t = \tanh(\mathbf{a}_t)$$

Figure 3: Detailed view of computations inside a single RNN cell. The input $\mathbf{x}_t$ and previous hidden state $\mathbf{h}_{t-1}$ are linearly transformed, summed with bias, and passed through the tanh activation to produce $\mathbf{h}_t$.

4.2 Common Loss Functions

Definition 4.2 (Mean Squared Error)**.** For regression tasks with continuous targets:

$$\mathcal{L}_t(\mathbf{y}_t, \hat{\mathbf{y}}_t) = \frac{1}{2}\|\mathbf{y}_t - \hat{\mathbf{y}}_t\|_2^2 = \frac{1}{2}\sum_{i=1}^{k}(y_t^{(i)} - \hat{y}_t^{(i)})^2 \tag{20}$$

Definition 4.3 (Cross-Entropy Loss)**.** For classification with softmax outputs:

$$\mathcal{L}_t(\mathbf{y}_t, \hat{\mathbf{y}}_t) = -\sum_{i=1}^{k}\hat{y}_t^{(i)}\log y_t^{(i)} = -\log y_t^{(c_t)} \tag{21}$$

where c_t is the true class index (assuming one-hot targets).

Proposition 4.4 (Softmax-CrossEntropy Gradient)**.** *When using softmax output with cross-entropy loss, the gradient simplifies to:*

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{o}_t} = \mathbf{y}_t - \hat{\mathbf{y}}_t \tag{22}$$

Proof. For one-hot target with true class c:

$$\begin{aligned}\frac{\partial \mathcal{L}_t}{\partial o_t^{(j)}} &= -\frac{\partial}{\partial o_t^{(j)}}\log\left(\frac{e^{o_t^{(c)}}}{\sum_i e^{o_t^{(i)}}}\right) \\ &= -\delta_{jc} + \frac{e^{o_t^{(j)}}}{\sum_i e^{o_t^{(i)}}} = -\hat{y}_t^{(j)} + y_t^{(j)}\end{aligned}$$

□

5 Backpropagation Through Time

Backpropagation Through Time (BPTT) is the algorithm for computing gradients in recurrent networks. It applies the chain rule to the unfolded computational graph.

5.1 Overview

The key insight is that the loss at time t depends on $\mathbf{h}_t$, which in turn depends on all previous hidden states through the recurrence. Thus, the gradient must be propagated backwards through time.

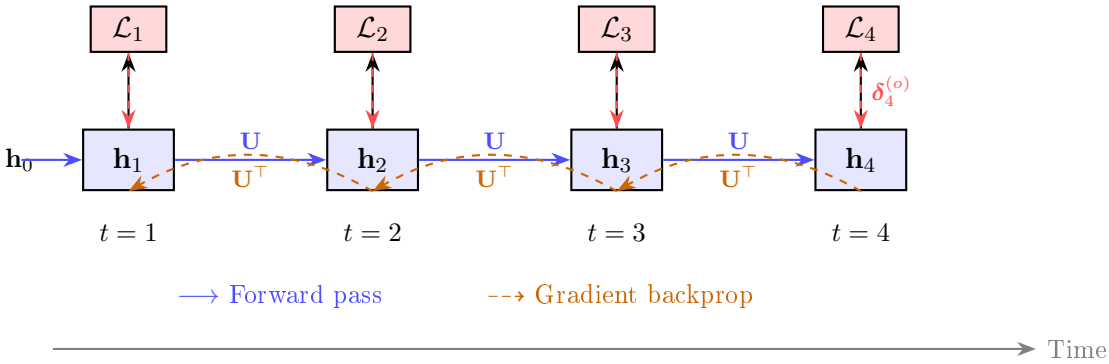


Figure 4: Gradient flow in BPTT. Forward pass (blue) propagates information forward through time. Backward pass (orange dashed) propagates gradients backward, multiplying by $\mathbf{U}^\top$ at each step. This multiplication causes gradients to vanish or explode over long sequences.

Theorem 5.1 (BPTT Principle)**.** *For the total loss $\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t$, the gradient with respect to any parameter θ is:*

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \theta} \tag{23}$$

where each $\frac{\partial \mathcal{L}_t}{\partial \theta}$ accounts for all paths through which θ influences $\mathcal{L}_t$.

5.2 Error Signal Definitions

Definition 5.2 (Output Error)**.** The error signal at the output layer is:

$$\boldsymbol{\delta}_t^{(o)} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{o}_t} \in \mathbb{R}^k \tag{24}$$

For softmax with cross-entropy: $\boldsymbol{\delta}_t^{(o)} = \mathbf{y}_t - \hat{\mathbf{y}}_t$.

Definition 5.3 (Hidden State Error)**.** The error signal at the hidden state is:

$$\boldsymbol{\delta}_t^{(h)} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \in \mathbb{R}^m \tag{25}$$

This accumulates contributions from the current output and future time steps.

5.3 BPTT Derivation

Theorem 5.4 (Hidden State Error Recurrence)**.** *The hidden state error satisfies the backward recurrence:*

$$\boxed{\boldsymbol{\delta}_t^{(h)} = \mathbf{V}^\top \boldsymbol{\delta}_t^{(o)} + \mathbf{U}^\top \boldsymbol{\delta}_{t+1}^{(h)} \odot (1 - \mathbf{h}_{t+1}^2) \cdot \mathbf{1}_{t<T}} \tag{26}$$

where $\odot$ denotes element-wise multiplication and $(1 - \mathbf{h}_{t+1}^2)$ is the derivative of tanh *evaluated at $\mathbf{a}_{t+1}$.*

Proof. The hidden state $\mathbf{h}_t$ influences the loss through two paths:

(i) Directly through $\mathbf{y}_t$: $\mathbf{h}_t \to \mathbf{o}_t \to \mathbf{y}_t \to \mathcal{L}_t$

(ii) Indirectly through $\mathbf{h}_{t+1}$: $\mathbf{h}_t \to \mathbf{a}_{t+1} \to \mathbf{h}_{t+1} \to \cdots \to \mathcal{L}$

For the direct path:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} = \mathbf{V}^\top \frac{\partial \mathcal{L}_t}{\partial \mathbf{o}_t} = \mathbf{V}^\top \boldsymbol{\delta}_t^{(o)}$$

For the indirect path (when $t < T$):

$$\begin{aligned} \left.\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t}\right|_{\text{indirect}} &= \frac{\partial \mathbf{a}_{t+1}}{\partial \mathbf{h}_t}^\top \frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{a}_{t+1}}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t+1}} \\ &= \mathbf{U}^\top \operatorname{diag}(1 - \mathbf{h}_{t+1}^2)\boldsymbol{\delta}_{t+1}^{(h)} \end{aligned}$$

Combining both paths yields the recurrence. □

Definition 5.5 (Pre-activation Error)**.** Define the error with respect to the pre-activation:

$$\boldsymbol{\delta}_t^{(a)} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}_t} = \boldsymbol{\delta}_t^{(h)} \odot (1 - \mathbf{h}_t^2) \tag{27}$$

5.4 Weight Gradients

Theorem 5.6 (BPTT Weight Gradients)**.** *The gradients with respect to the weight matrices are:*

$$\frac{\partial \mathcal{L}}{\partial \mathbf{V}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(o)} \mathbf{h}_t^\top \tag{28}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(a)} \mathbf{x}_t^\top \tag{29}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{U}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(a)} \mathbf{h}_{t-1}^\top \tag{30}$$

Proof. For $\mathbf{V}$: At each time step t, we have $\mathbf{o}_t = \mathbf{V}\mathbf{h}_t + \mathbf{c}$, so:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{V}} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{o}_t} \frac{\partial \mathbf{o}_t}{\partial \mathbf{V}} = \boldsymbol{\delta}_t^{(o)} \mathbf{h}_t^\top$$

For $\mathbf{W}$ and $\mathbf{U}$: Since $\mathbf{a}_t = \mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}$:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{W}} &= \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \mathbf{a}_t} \frac{\partial \mathbf{a}_t}{\partial \mathbf{W}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(a)} \mathbf{x}_t^\top \\ \frac{\partial \mathcal{L}}{\partial \mathbf{U}} &= \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \mathbf{a}_t} \frac{\partial \mathbf{a}_t}{\partial \mathbf{U}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(a)} \mathbf{h}_{t-1}^\top \end{aligned}$$

□

Theorem 5.7 (BPTT Bias Gradients)**.** *The gradients with respect to the bias vectors are:*

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(o)} \tag{31}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(a)} \tag{32}$$

5.5 Complete BPTT Algorithm

BPTT Algorithm

Forward Pass:

1. Initialize $\mathbf{h}_0 = \mathbf{0}$
2. For $t = 1$ to T:
   - $\mathbf{a}_t = \mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}$
   - $\mathbf{h}_t = \tanh(\mathbf{a}_t)$
   - $\mathbf{o}_t = \mathbf{V}\mathbf{h}_t + \mathbf{c}$
   - $\mathbf{y}_t = \text{softmax}(\mathbf{o}_t)$
   - Compute $\mathcal{L}_t$

Backward Pass:

1. Initialize $\boldsymbol{\delta}_{T+1}^{(h)} = \mathbf{0}$
2. For $t = T$ down to 1:
   - $\boldsymbol{\delta}_t^{(o)} = \mathbf{y}_t - \hat{\mathbf{y}}_t$
   - $\boldsymbol{\delta}_t^{(h)} = \mathbf{V}^\top \boldsymbol{\delta}_t^{(o)} + \mathbf{U}^\top(\boldsymbol{\delta}_{t+1}^{(h)} \odot (1 - \mathbf{h}_{t+1}^2))$
   - $\boldsymbol{\delta}_t^{(a)} = \boldsymbol{\delta}_t^{(h)} \odot (1 - \mathbf{h}_t^2)$
   - Accumulate gradients

6 The Vanishing and Exploding Gradient Problem

This section presents the central pathology of vanilla RNNs that motivates gated architectures.

6.1 Gradient Flow Analysis

Theorem 6.1 (Long-Range Gradient). *The gradient of the loss at time t with respect to the hidden state at time $\tau < t$ is:*

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \prod_{i=\tau+1}^{t} \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} \tag{33}$$

Proof. By repeated application of the chain rule along the path $\mathbf{h}_\tau \to \mathbf{h}_{\tau+1} \to \cdots \to \mathbf{h}_t \to \mathcal{L}_t$:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \cdots \frac{\partial \mathbf{h}_{\tau+1}}{\partial \mathbf{h}_\tau} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \prod_{i=\tau+1}^{t} \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}}$$

□

Lemma 6.2 (Jacobian of Hidden State). *The Jacobian of the hidden state transition is:*

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \text{diag}(1 - \mathbf{h}_t^2)\mathbf{U} \tag{34}$$

where $\text{diag}(1 - \mathbf{h}_t^2)$ *is a diagonal matrix with entries* $(1 - h_t^{(j)2})$.

Proof. Since $\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b})$:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \frac{\partial \mathbf{h}_t}{\partial \mathbf{a}_t}\frac{\partial \mathbf{a}_t}{\partial \mathbf{h}_{t-1}} = \operatorname{diag}(1 - \mathbf{h}_t^2)\mathbf{U}$$

□

6.2 The Vanishing Gradient Problem

Theorem 6.3 (Vanishing Gradient). *Let $\gamma = \|\mathbf{U}\|$ be the spectral norm of $\mathbf{U}$. If $\gamma < 1$, then:*

$$\left\|\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau}\right\| \leq \left\|\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t}\right\| \gamma^{t-\tau} \tag{35}$$

*Thus, gradients decay **exponentially** with the time gap $(t - \tau)$.*

Proof. Using the submultiplicativity of matrix norms:

$$\begin{aligned}\left\|\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau}\right\| &= \left\|\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t}\prod_{i=\tau+1}^{t} \operatorname{diag}(1 - \mathbf{h}_i^2)\mathbf{U}\right\| \\ &\leq \left\|\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t}\right\| \prod_{i=\tau+1}^{t} \|\operatorname{diag}(1 - \mathbf{h}_i^2)\|\|\mathbf{U}\|\end{aligned}$$

Since $|1 - h_i^{(j)2}| \leq 1$ for all components (as tanh outputs are in $(-1, 1)$):

$$\|\operatorname{diag}(1 - \mathbf{h}_i^2)\| \leq 1$$

Therefore:

$$\left\|\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau}\right\| \leq \left\|\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t}\right\| \gamma^{t-\tau}$$

□

Corollary 6.4 (Exponential Decay). *For a gap of $k = t - \tau$ time steps with $\gamma = 0.9$:*

k	*10*	*50*	*100*	*200*
γ^k	*0.35*	5×10^{-3}	2.7×10^{-5}	7×10^{-10}

Gradients become negligible after only a few dozen time steps.

6.3 The Exploding Gradient Problem

Theorem 6.5 (Exploding Gradient). *If $\gamma = \|\mathbf{U}\| > 1$ and the Jacobian products do not shrink sufficiently, gradients can grow exponentially:*

$$\left\|\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau}\right\| \geq C \cdot \gamma^{t-\tau} \tag{36}$$

for some constant $C > 0$ depending on the activation saturation.

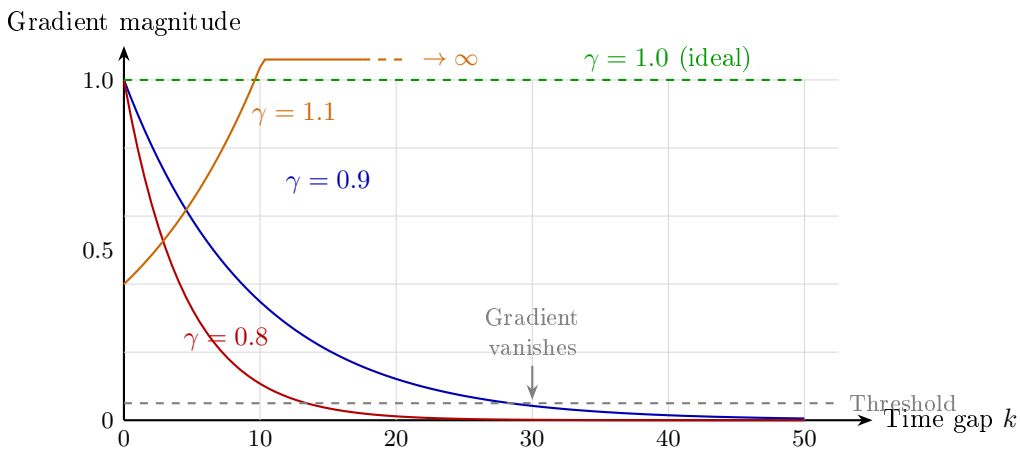


Figure 5: Visualization of gradient magnitude decay/growth as a function of the time gap k. When $\gamma < 1$ (blue, red curves), gradients vanish exponentially. When $\gamma > 1$ (orange), gradients explode. Only $\gamma = 1$ (green dashed) preserves gradient magnitude, but this is unstable in practice.

Consequences of Gradient Pathologies

Vanishing Gradients ($\gamma < 1$):

- Long-range dependencies cannot be learned
- Network only captures short-term patterns
- Weights for distant inputs receive negligible updates

Exploding Gradients ($\gamma > 1$):

- Training becomes unstable
- Loss oscillates or diverges
- Numerical overflow in computations

6.4 Formal Characterization

Theorem 6.6 (Necessary Condition for Gradient Preservation)**.** *For gradients to neither vanish nor explode, we require:*

$$\prod_{i=\tau+1}^{t} \| \operatorname{diag}(1 - \mathbf{h}_i^2)\mathbf{U}\| \approx 1 \tag{37}$$

for all relevant time gaps. This is difficult to achieve with fixed weights.

Proposition 6.7 (Eigenvalue Analysis)**.** *Let $\lambda_1, \ldots, \lambda_m$ be the eigenvalues of $\mathbf{U}$. The gradient behavior is characterized by:*

- *If $|\lambda_{\max}| < 1$: Vanishing gradients dominate*
- *If $|\lambda_{\max}| > 1$: Exploding gradients risk*
- *If $|\lambda_{\max}| = 1$: Marginally stable (ideal but unstable)*

7 Mitigation Strategies

7.1 Gradient Clipping

Definition 7.1 (Gradient Clipping)**.** Gradient clipping rescales the gradient when its norm exceeds a threshold θ:

$$\tilde{\mathbf{g}} = \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\| \leq \theta \\ \frac{\theta}{\|\mathbf{g}\|}\mathbf{g} & \text{if } \|\mathbf{g}\| > \theta \end{cases} \tag{38}$$

This addresses exploding gradients but does not solve vanishing gradients.

7.2 Orthogonal Initialization

Proposition 7.2 (Orthogonal Matrix Property)**.** *If* $\mathbf{U}$ *is orthogonal (*$\mathbf{U}^\top\mathbf{U} = \mathbf{I}$*), then* $\|\mathbf{U}\| = 1$ *(all singular values equal 1).*

Initializing $\mathbf{U}$ as an orthogonal matrix helps maintain gradient magnitude, but this property is not preserved during training.

7.3 Truncated BPTT

Definition 7.3 (Truncated BPTT)**.** Truncated BPTT limits backpropagation to the most recent k time steps:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau} \approx \begin{cases} \text{computed} & \text{if } t - \tau \leq k \\ 0 & \text{if } t - \tau > k \end{cases} \tag{39}$$

This is a practical compromise that accepts the inability to learn very long-range dependencies.

7.4 Architectural Solutions: Gated RNNs

The fundamental solution to the vanishing gradient problem is to redesign the architecture itself. The key insight is to create additive paths for gradient flow.

Remark 7.4 (Preview of LSTM)**.** The Long Short-Term Memory (LSTM) architecture addresses the vanishing gradient problem by:

(i) Introducing an additive cell state update: $_t =_t \odot_{t-1} +_t \tilde{\odot}_t$

(ii) Using learnable gates to control information flow

(iii) Creating a "gradient highway" where $\frac{\partial_t}{\partial_{t-1}} = \text{diag}(_t) \approx \mathbf{I}$

This transforms the multiplicative gradient decay into a controllable, potentially additive process.

8 RNN Variants

8.1 Jordan Network

Definition 8.1 (Jordan Network)**.** The Jordan network feeds the output (rather than hidden state) back to the input:

$$\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{y}_{t-1} + \mathbf{b}) \tag{40}$$

$$\mathbf{y}_t = g(\mathbf{V}\mathbf{h}_t + \mathbf{c}) \tag{41}$$

8.2 Deep RNN

Definition 8.2 (Deep (Stacked) RNN)**.** A deep RNN with L layers:

$$\mathbf{h}_t^{(1)} = \tanh(\mathbf{W}^{(1)}\mathbf{x}_t + \mathbf{U}^{(1)}\mathbf{h}_{t-1}^{(1)} + \mathbf{b}^{(1)}) \tag{42}$$

$$\mathbf{h}_t^{(\ell)} = \tanh(\mathbf{W}^{(\ell)}\mathbf{h}_t^{(\ell-1)} + \mathbf{U}^{(\ell)}\mathbf{h}_{t-1}^{(\ell)} + \mathbf{b}^{(\ell)}), \quad \ell = 2, \ldots, L \tag{43}$$

8.3 Bidirectional RNN

Definition 8.3 (Bidirectional RNN)**.** Processes sequences in both directions:

$$\overrightarrow{\mathbf{h}}_t = \tanh(\mathbf{W}\mathbf{x}_t + \overrightarrow{\mathbf{U}}\,\overrightarrow{\mathbf{h}}_{t-1} + \mathbf{b}) \tag{44}$$

$$\overleftarrow{\mathbf{h}}_t = \tanh(\mathbf{W}\mathbf{x}_t + \overleftarrow{\mathbf{U}}\,\overleftarrow{\mathbf{h}}_{t+1} + \mathbf{b}) \tag{45}$$

$$\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \tag{46}$$

9 Conclusion

This document has presented a rigorous mathematical treatment of Recurrent Neural Networks. The key findings are:

1. **Architecture**: The RNN maintains a hidden state $\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b})$ that theoretically captures the entire sequence history.
2. **Learning**: The BPTT algorithm computes gradients by unfolding the recurrence and applying the chain rule backwards through time.
3. **Fundamental Limitation**: The gradient $\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_\tau}$ involves a product of $(t - \tau)$ Jacobian matrices, leading to exponential decay or growth.
4. **Consequence**: Vanilla RNNs cannot effectively learn dependencies spanning more than approximately 10-20 time steps.

The vanishing gradient problem is not merely a numerical inconvenience but a fundamental limitation of the vanilla RNN architecture. This motivates the development of gated architectures—particularly the Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU)—which are treated in the companion document.

References

[1] Elman, J. L. (1990). *Finding structure in time.* Cognitive Science, 14(2), 179–211.

[2] Jordan, M. I. (1986). *Serial order: A parallel distributed processing approach.* Technical Report 8604, Institute for Cognitive Science, UC San Diego.

[3] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). *Learning representations by back-propagating errors.* Nature, 323(6088), 533–536.

[4] Bengio, Y., Simard, P., & Frasconi, P. (1994). *Learning long-term dependencies with gradient descent is difficult.* IEEE Transactions on Neural Networks, 5(2), 157–166.

[5] Hochreiter, S. (1991). *Untersuchungen zu dynamischen neuronalen Netzen.* Diploma thesis, TU Munich.

[6] Pascanu, R., Mikolov, T., & Bengio, Y. (2013). *On the difficulty of training recurrent neural networks.* In Proceedings of ICML.

[7] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning.* MIT Press.